XS WALLET

Technical Specification Document

HD Wallet + Atomic Swaps Desktop Application

Document	Value
Version	0.1.0
Status	Draft - Foundation Complete
Date	January 2026
Classification	Technical Specification
Target Audience	Developers, Architects, Auditors

Table of Contents

1. Executive Summary

XS Wallet is a self-custody desktop application enabling atomic swaps between Bitcoin (BTC), Liquid (L-BTC), and Lightning Network (LN) using Taproot and the Boltz API v2 as liquidity provider. The system maintains true self-custody while providing seamless cross-chain and cross-layer swaps without requiring users to operate their own swap infrastructure.

1.1 Core Capabilities

Capability	Implementation	Standard/Protocol
HD Wallet	Hierarchical deterministic key derivation	BIP39, BIP32, BIP84, BIP85
Atomic Swaps	Hash Time-Locked Contracts (HTLC)	Boltz API v2, Taproot P2TR
MuSig2 Signing	Aggregated Schnorr signatures	BIP340, MuSig2 draft
Encryption	Memory-hard KDF + authenticated encryption	Argon2id, AES-256-GCM
Persistence	Embedded SQL database with WAL	SQLite 3.31+
Node Integration	Embedded full nodes with verified binaries	Bitcoin Core, Elements, LND

Table 1.1: Core Capabilities Matrix

1.2 Design Principles

Self-Custody: User controls all private keys; no server holds funds or signing authority.

Zero Trust: All external data (Boltz responses, node data) is verified locally before use.

Crash Recovery: All swap states are persisted with idempotent operations; swaps can resume after any failure.

Deterministic Restoration: Pending swaps can be restored from mnemonic alone using BIP85 child seeds.

2. System Architecture

2.1 Component Overview

The system is structured as a multi-process Electron application with crash isolation between UI, backend logic, and embedded nodes. This architecture ensures that a crash in any component does not affect the others, and swap operations can continue even if the UI restarts.

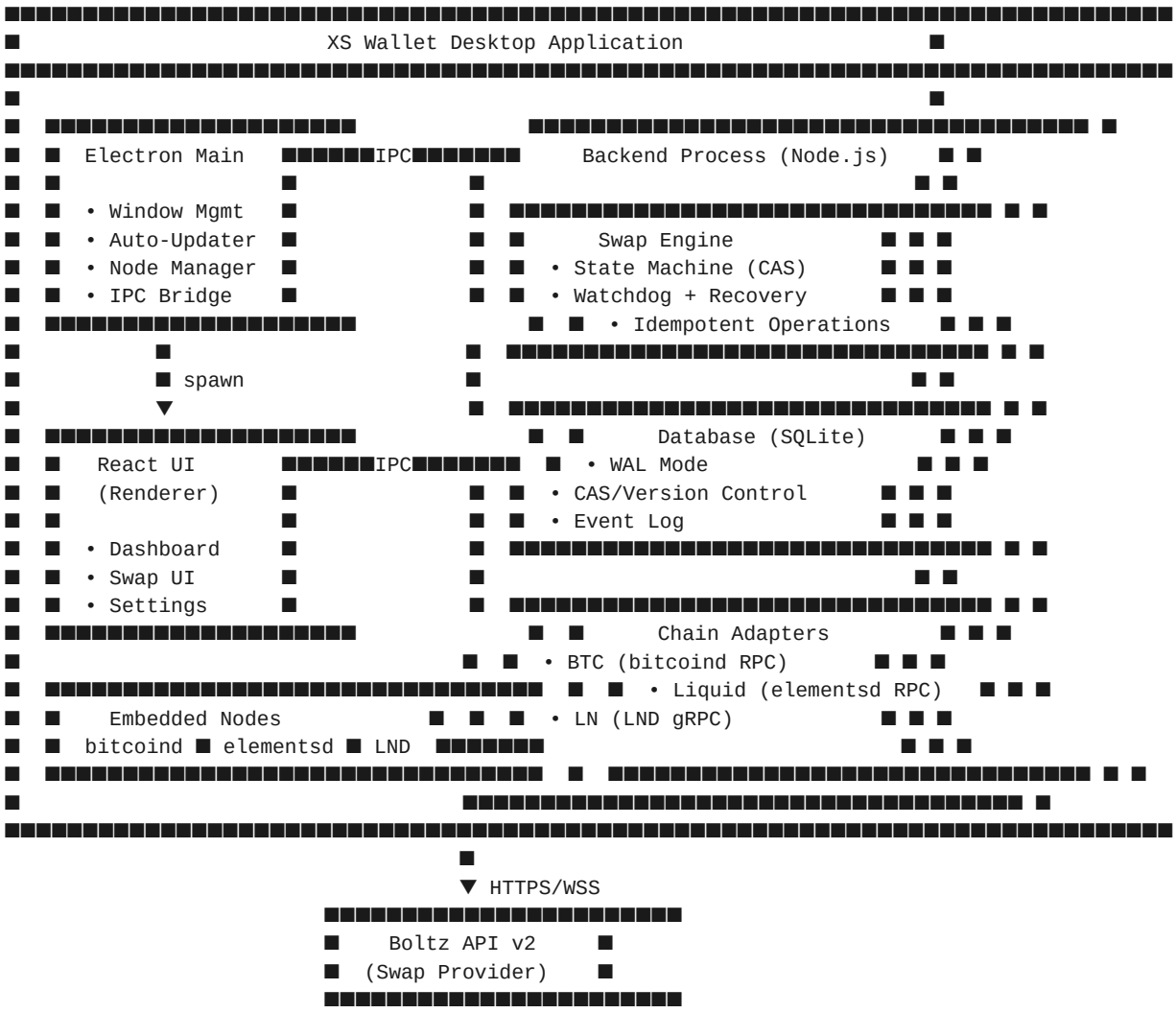


Figure 2.1: System Architecture Diagram

2.2 Process Model

Process	Role	Isolation Benefit
Main (Electron)	Window management, IPC routing, node spawning	Core stability; manages lifecycle
Backend (Node.js)	Swap engine, database, adapters, Boltz client	UI crash does not affect swaps
Renderer (Chromium)	React UI, user interactions	Sandboxed; minimal privileges
bitcoind	Bitcoin full node (RPC server)	Separate process; own data dir
elementsd	Liquid/Elements node (RPC server)	Separate process; own data dir

LND	Lightning Network daemon (gRPC server)	Separate process; macaroon auth

Table 2.1: Process Model

2.3 Data Flow

All user-initiated operations flow through the IPC bridge to the backend process. The backend maintains authoritative state in SQLite and coordinates with chain adapters and Boltz API. Critical invariant: **WebSocket events from Boltz are triggers only; on-chain/LND state is always verified before acting.**

3. Database Specification

3.1 Schema Design

The database uses SQLite with Write-Ahead Logging (WAL) for optimal read/write concurrency. All swap state mutations use Compare-And-Swap (CAS) with version-based optimistic locking to prevent race conditions without blocking queries.

3.1.1 Critical Pragmas

Pragma	Value	Rationale
journal_mode	WAL	Concurrent reads during writes; crash recovery
synchronous	NORMAL	Balance durability/performance (safe with WAL)
busy_timeout	5000	Wait 5s for locks; prevents "database is locked"
foreign_keys	ON	Enforce referential integrity
temp_store	MEMORY	Temp tables in RAM for performance
cache_size	-20000	~20MB page cache

Table 3.1: SQLite Pragmas

3.2 Table Definitions

3.2.1 swaps - Authoritative Swap State

Column	Type	Constraints	Description
id	TEXT	PRIMARY KEY	Unique swap identifier (UUID)
kind	TEXT	CHECK(IN submarine,reverse,chain)	Swap type
env	TEXT	CHECK(IN regtest,testnet,mainnet)	Network environment
version	INTEGER	NOT NULL DEFAULT 0	CAS version for optimistic locking
state	TEXT	CHECK(IN open,locked,...)	Current state machine state
locked_intent	TEXT (JSON)	NOT NULL when state != open	Immutable quote/fees snapshot
swap_key_index	INTEGER	NOT NULL	BIP85 derivation index
preimage_hash_hex	TEXT	CHECK(length=64)	SHA256 hash of preimage R
claim_pubkey_hex	TEXT	CHECK(length IN 64,66)	User claim public key
refund_pubkey_hex	TEXT	CHECK(length IN 64,66)	User refund public key
musig_session_id	TEXT		MuSig2 session identifier
lockup_txid	TEXT	CHECK(length=64)	Funding transaction ID
lockup_amount_sat	TEXT	CHECK(GLOB [0-9]*)	Amount in satoshis (bigint as string)

created_at	TEXT	DEFAULT ISO8601	Creation timestamp
updated_at	TEXT	DEFAULT ISO8601	Last update timestamp

Table 3.2: swaps Table (partial - 40+ columns total)

***Invariant**: When state is not in (open, failed, canceled), the locked_intent field MUST be non-null. This is enforced via CHECK constraint and ensures every active swap has an immutable record of agreed parameters.*

3.2.2 Supporting Tables

Table	Primary Key	Purpose
swap_events	seq (AUTOINCREMENT)	Immutable audit log; enables replay/debug
swap_ops	(swap_id, op_key)	Idempotent operation ledger; prevents duplicate broadcasts
utxo_reservations	(chain, txid, vout)	Prevents UTXO double-spend across swaps
ln_reservations	payment_hash_hex	Prevents duplicate LN payments
app_config	key	User configuration; snapshot captured in locked_intent

Table 3.3: Supporting Tables

3.3 Concurrency Model

All state transitions use Compare-And-Swap (CAS) with atomic transactions:

```
-- CAS Update Pattern
UPDATE swaps
SET state = 'locked',
    version = version + 1,
    updated_at = strftime('%Y-%m-%dT%H:%M:%fZ', 'now'),
    locked_intent = ?
WHERE id = ? AND version = ? AND state = 'open'
RETURNING version, state;

-- If RETURNING is empty: version mismatch (concurrent modification)
-- Application must reload and retry or abort
```

Listing 3.1: CAS Update Pattern

4. Atomic Swap Protocol

4.1 Submarine Swap (On-chain → Lightning)

Submarine swaps convert on-chain funds (BTC or L-BTC) to Lightning capacity. The user locks funds in a P2TR HTLC; Boltz pays the user's LN invoice and claims the HTLC using the revealed preimage.

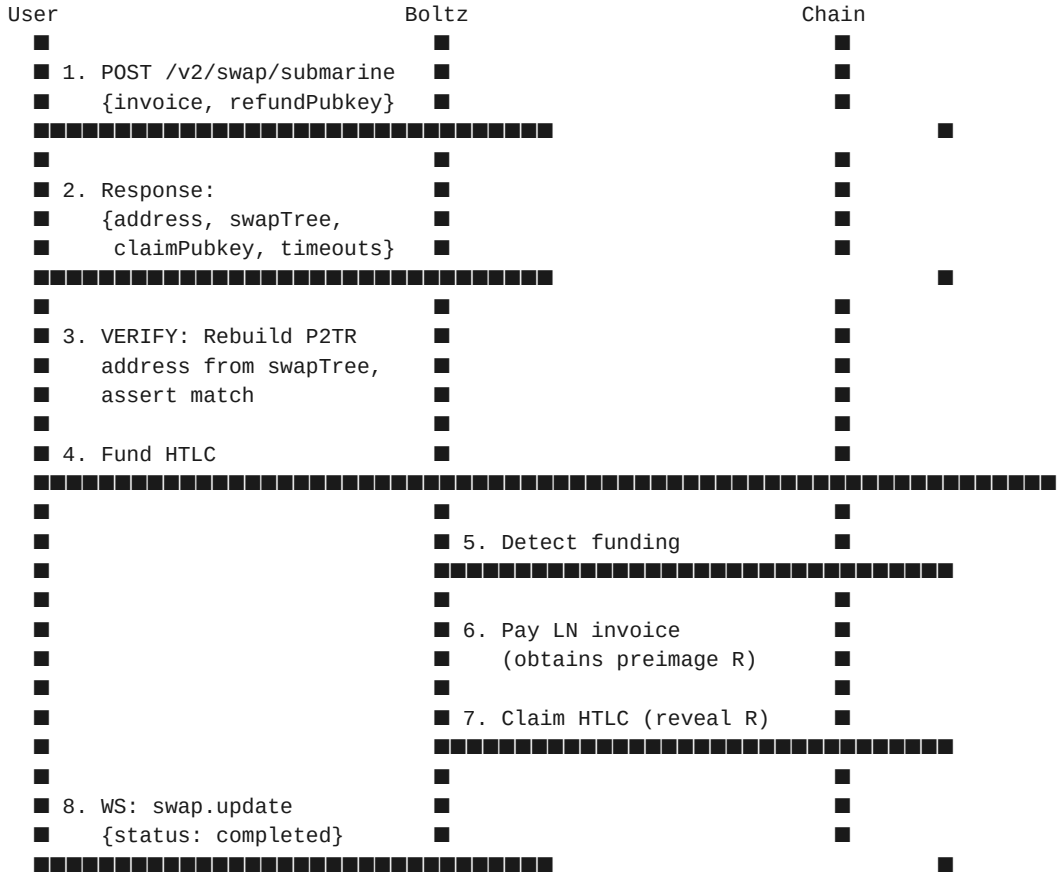
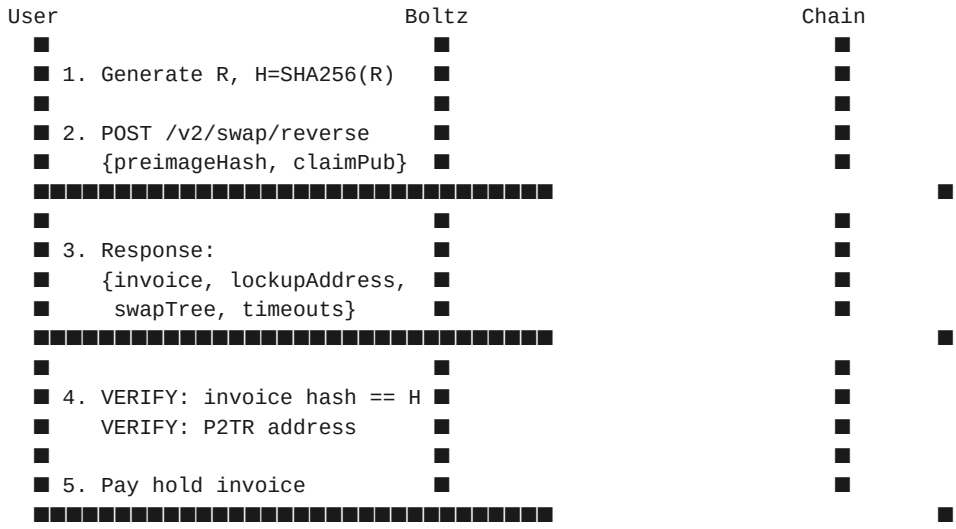


Figure 4.1: Submarine Swap Sequence

4.2 Reverse Swap (Lightning → On-chain)

Reverse swaps convert Lightning balance to on-chain funds. The user generates preimage R and hash H=SHA256(R), pays a Boltz hold invoice, and claims the on-chain HTLC by revealing R.



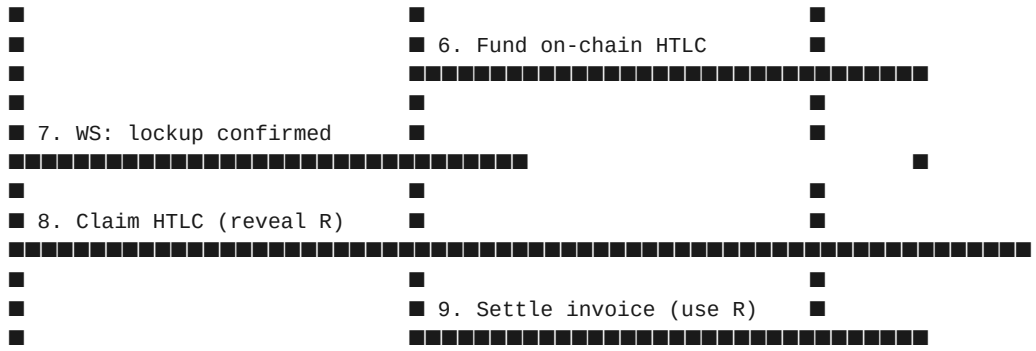


Figure 4.2: Reverse Swap Sequence

4.3 Chain Swap (BTC ↔ Liquid)

Chain swaps are atomic cross-chain swaps between Bitcoin mainchain and Liquid sidechain. Timeouts are staggered (T_liquid < T_btc) to prevent race conditions.

4.4 State Machine

State	Description	Transitions
open	Initial state; quote accepted but not locked	locked | canceled
locked	Parameters locked; intent immutable	commit_started | canceled
commit_started	Funding transaction broadcast	waiting | failed
waiting	Awaiting counterparty action	waiting_claim_details | refund_coop_waiting
waiting_claim_details	Awaiting claim transaction details	signing_musig2_partial
signing_musig2_partial	Creating MuSig2 partial signature	sent_partial_to_provider
sent_partial_to_provider	Partial sig sent; awaiting broadcast	waiting_provider_broadcast
waiting_provider_broadcast	Provider broadcasting claim tx	completed | fallback_script_ready
refund_coop_waiting	Attempting cooperative refund	completed | fallback_script_ready
fallback_script_ready	Preparing script-path claim/refund	refunding | completed
refunding	Refund tx broadcast; awaiting confirmation	completed | failed
completed	Terminal success state	(terminal)
failed	Terminal failure state	(terminal)
canceled	Canceled before funding	(terminal)

Table 4.1: Swap State Machine

5. Cryptographic Specifications

5.1 Key Derivation Hierarchy

```
Master Seed (BIP39 - 24 words)
■
■■■ BIP32 Master Key (m)
■   ■
■   ■■■ BIP84 (Native SegWit Wallet)
■   ■   ■■■ m/84'/0'/0'/...  (regular wallet addresses)
■   ■
■   ■■■ BIP85 Child Seed (Swap Subtree)
■       ■
■       ■■■ m/0/index  (swap key pairs)
■           ■
■           ■■■ preimage R = SHA256(privateKey)
■
Restoration: Given mnemonic → derive swap keys → reconstruct pending swaps
```

Figure 5.1: Key Derivation Hierarchy

5.2 Vault Encryption

Parameter	Value	Rationale
KDF	Argon2id	Memory-hard; resistant to GPU/ASIC attacks
Memory	64 MB	Balance security/performance for desktop
Iterations	3	Standard recommendation
Parallelism	1	Single-threaded derivation
Salt Length	16 bytes	Random per-wallet
Cipher	AES-256-GCM	Authenticated encryption
IV Length	12 bytes	GCM standard
Tag Length	16 bytes	Full authentication tag

Table 5.1: Vault Encryption Parameters

***Offline Attack Mitigation**: Rate limiting (exponential backoff) protects runtime but not offline database theft. For enhanced protection, derive final key from PIN + device secret stored in OS keychain (Keychain/DPAPI/SecretService). MVP uses PIN-only; device secret is v1.1.*

5.3 Taproot + MuSig2

Swap HTLCs use P2TR (Pay-to-Taproot) with:

Component	Specification
Key Aggregation Order	Provider pubkey first, then user pubkey (deterministic)
Nonce Generation	Deterministic from session ID + private key

Session Persistence	musig_* fields in swaps table; survives restart
Key-Path Spend	MuSig2 aggregated signature (cooperative claim/refund)
Script-Path Fallback	HTLC conditions if cooperation fails (timeout-based)

Table 5.2: MuSig2 Specification

6. Node Management

6.1 Lifecycle Management

The Node Manager handles binary verification, spawning, health monitoring, and graceful shutdown of embedded nodes. Each node has isolated data directories per network.

Node	Version	Data Directory	Health Check
bitcoind	26.0	%APPDATA%/xs-wallet/nodes/btc/{network}	getblockchaininfo RPC
elementsd	23.2.1	%APPDATA%/xs-wallet/nodes/liquid/{network}	getblockchaininfo RPC
LND	0.17.3	%APPDATA%/xs-wallet/nodes/lnd/{network}	getinfo gRPC

Table 6.1: Node Specifications

6.2 Binary Verification

Binaries are downloaded on first run from official sources (GitHub Releases, bitcoincore.org) and verified using SHA256 checksums from a signed manifest.

> ***Trust Chain**: Application embeds a pinned public key. On startup, downloads nodes-manifest.json from GitHub Releases, verifies signature against pinned key, then verifies each binary checksum against manifest. No central custody server; only public distribution endpoints.*

7. Security Model

7.1 Threat Model

Threat	In Scope	Mitigation
Seed theft (offline)	Yes	Argon2id + device secret (v1.1)
PIN brute force (runtime)	Yes	Exponential backoff; lockout after 10 attempts
Swap manipulation	Yes	Local script verification; never trust provider
Supply chain attack	Yes	Signed manifest; SHA256 verification
Physical device access	No (future)	Consider hardware wallet support
OS compromise (root)	No	Out of scope; assume trusted OS

Table 7.1: Threat Model

7.2 Cryptographic Verification

Verification	When	Failure Action
P2TR address rebuild	Before funding HTLC	Abort swap; log error
Invoice hash == preimage hash	Before paying hold invoice	Abort swap; log error
Timeout sanity (T_liquid < T_btc)	Before locking	Abort swap; log error
MuSig2 partial signature	Before sending to provider	Abort; attempt script-path
On-chain confirmation	After claim/refund broadcast	Retry with higher fee

Table 7.2: Verification Points

8. API Specifications

8.1 Boltz API v2 Endpoints

Endpoint	Method	Purpose
/v2/swap/submarine	POST	Create submarine swap (on-chain → LN)
/v2/swap/reverse	POST	Create reverse swap (LN → on-chain)
/v2/swap/chain	POST	Create chain swap (BTC ↔ Liquid)
/v2/swap/{id}	GET	Get swap status
/v2/swap/{id}/claim	POST	Submit claim transaction details
/v2/swap/{id}/refund	POST	Submit refund transaction details
/v2/ws	WebSocket	Real-time swap status updates
/v2/nodes	GET	Get routing hints for LN payments

Table 8.1: Boltz API v2 Endpoints

8.2 Internal IPC API

Channel	Direction	Payload
swap:create	Renderer → Backend	{kind, fromAsset, toAsset, amount}
swap:status	Backend → Renderer	{swapId, state, details}
wallet:balance	Renderer → Backend	{chain: btc|liquid|ln}
wallet:balance:response	Backend → Renderer	{confirmed, unconfirmed, pending}
node:status	Backend → Renderer	{bitcoind, elementsd, lnd: running|stopped|syncing}

Table 8.2: IPC API Channels

9. Implementation Roadmap

Phase	Duration	Deliverables	Status
Foundation	Complete	Schema, DB Layer, Node Manager, Decisions	Done
Backend Core	2 weeks	Key Vault, Adapters, Boltz Client, Swap Engine	Next
Frontend	1.5 weeks	React UI, Onboarding, Dashboard, Swap Interface	Pending
Electron	1 week	Main Process, IPC, Auto-update, Deep Links	Pending
Packaging	0.5 weeks	Installers, Code Signing, E2E Tests	Pending

Table 9.1: Implementation Roadmap

9.1 Post-MVP Roadmap

Version	Target	Features
v1.1	Q2 2026	Hardware wallet support, SQLCipher, device secret
v1.2	Q3 2026	Coinjoin integration, Payjoin, LNURL
v2.0	Q4 2026	RGB assets, Taproot Assets, DLC support

Table 9.2: Post-MVP Roadmap